

CA1: Dataframe Manipulation with Spotify Data

Introduction

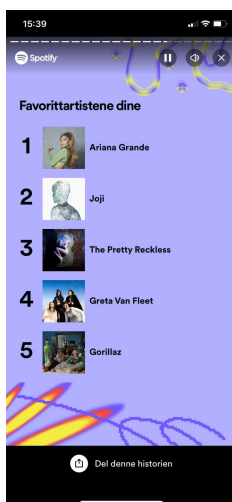
Pandas is an extremely powerful tool to handle large amounts of tabular data. In this compulsory assignment, you will use Pandas to explore one of the TA's personal spotify data in depth.

Additional information:

- Feel free to create additional code cells if you feel that one cell per subtask is not sufficient.
- Remember, Pandas uses very efficient code to handle large amounts of data. For-loops are not efficient. If you ever have to use a for-loop to loop over the rows in the DataFrame, you have *probably* done something wrong.
- Label all graphs and charts if applicable.

Task

I typically enjoy indie and rock music. I am a big fan of everything from old-fashioned rock and roll like Led Zeppelin and Jimi Hendrix, to newer indie artists like Joji and Lana Del Rey. This is why my spotify wrapped for 2023 came as quite a surprise:



Now, I'm no hater of pop music, but this was unexpected. For this assignment, you will investigate my listening habits, including a deep dive into my Ariana Grande listening habits, and try to find an answer to why she was my top artist; was there a fault in the spotify

algorithm? Am I actually secretly an *Arianator*? (yes, I did have to look that up). Or am I just lying to myself about how often I listen to guilty pleasure music?

Part 1: Initial loading and exploration

1.0 Import necessary libraries:

pandas, numpy, matplotlib.pyplot (other libraries such as seaborn or plotly are also allowed if you want prettier plots). It might also be a good idea to use **os** for task 2.0

```
In [136... import matplotlib.pyplot as plt
import pandas as pd
```

1.1 Loading the data

Load the dataset in the file `streaming_history_0.csv` into a Pandas DataFrame called `df_spotify_0`.

```
In [137... df_spotify_0 = pd.read_csv("spotify_data/streaminghistory0.csv")
```

1.2 Help function

Use the Python command `help` to help you understand how to use the `pd.DataFrame.head` and `pd.DataFrame.tail` methods.

```
In [138... help(pd.DataFrame.head)
```

Help on function head in module pandas.core.generic:

```
head(self, n: 'int' = 5) -> 'Self'
    Return the first `n` rows.
```

This function returns the first `n` rows for the object based on position. It is useful for quickly testing if your object has the right type of data in it.

For negative values of `n`, this function returns all rows except the last `|n|` rows, equivalent to ``df[:n]``.

If n is larger than the number of rows, this function returns all rows.

Parameters

n : int, default 5
Number of rows to select.

Returns

same type as caller
The first `n` rows of the caller object.

See Also

DataFrame.tail: Returns the last `n` rows.

Examples

```
>>> df = pd.DataFrame({'animal': ['alligator', 'bee', 'falcon', 'lion',
...                               'monkey', 'parrot', 'shark', 'whale', 'zebra']})
>>> df
   animal
0  alligator
1      bee
2   falcon
3     lion
4   monkey
5   parrot
6    shark
7   whale
8   zebra
```

Viewing the first 5 lines

```
>>> df.head()
   animal
0  alligator
1      bee
2   falcon
3     lion
4   monkey
```

Viewing the first `n` lines (three in this case)

```
>>> df.head(3)
      animal
0  alligator
1       bee
2     falcon
```

For negative values of `n`

```
>>> df.head(-3)
      animal
0  alligator
1       bee
2     falcon
3       lion
4     monkey
5     parrot
```

In [139... `help(pd.DataFrame.tail)`

Help on function tail in module pandas.core.generic:

```
tail(self, n: 'int' = 5) -> 'Self'
    Return the last `n` rows.
```

This function returns last `n` rows from the object based on position. It is useful for quickly verifying data, for example, after sorting or appending rows.

For negative values of `n`, this function returns all rows except the first `|n|` rows, equivalent to ``df[|n|:]``.

If n is larger than the number of rows, this function returns all rows.

Parameters

n : int, default 5
Number of rows to select.

Returns

type of caller
The last `n` rows of the caller object.

See Also

DataFrame.head : The first `n` rows of the caller object.

Examples

```
>>> df = pd.DataFrame({'animal': ['alligator', 'bee', 'falcon', 'lion',
...                               'monkey', 'parrot', 'shark', 'whale', 'zebra']})
>>> df
   animal
0  alligator
1      bee
2   falcon
3     lion
4   monkey
5   parrot
6    shark
7   whale
8    zebra
```

Viewing the last 5 lines

```
>>> df.tail()
   animal
4  monkey
5  parrot
6  shark
7  whale
8  zebra
```

Viewing the last `n` lines (three in this case)

```
>>> df.tail(3)
      animal
6  shark
7  whale
8  zebra
```

For negative values of `n`

```
>>> df.tail(-3)
      animal
3  lion
4  monkey
5  parrot
6  shark
7  whale
8  zebra
```

1.3 Getting an overview

Print the first **five** and last **ten** rows of the dataframe. Have a quick look at which columns are in the dataset.

```
In [140... df_spotify_0.head()
```

```
Out[140...
      endTime      artistName      trackName  msPlayed
0  2022-12-03 02:02  Cigarettes After Sex      Truly    30000.0
1  2022-12-03 02:02    Leonard Cohen  Take This Waltz - Paris Version    8210.0
2  2022-12-06 21:05    Vlad Holiday    So Damn Into You    37895.0
3  2022-12-06 21:05        Lorde      Team    8984.0
4  2022-12-06 21:05    Ariana Grande    Into You    1221.0
```

```
In [141... df_spotify_0.tail(10)
```

Out[141...

	endTime	artistName	trackName	msPlayed
11949	2023-01-02 20:58	Ariana Grande	six thirty	1699.0
11950	2023-01-02 20:58	Leonard Cohen	Thanks for the Dance	19483.0
11951	2023-01-02 20:59	Des Rocs	Used to the Darkness	185.0
11952	2023-01-02 20:59	Caroline Polachek	Hit Me Where It Hurts	603.0
11953	2023-01-02 20:59	Caroline Polachek	Hit Me Where It Hurts	208.0
11954	2023-01-02 20:59	Kaizers Orchestra	Resistansen	208.0
11955	2023-01-02 20:59	Mr.Kitty	After Dark	101447.0
11956	2023-01-02 20:59	daddy's girl	after dark x sweater weather	301.0
11957	2023-01-02 20:59	daddy's girl	after dark x sweater weather	208.0
11958	2023-01-02 20:59	daddy's girl	after dark x sweater weather	789.0

1.4 Formatting correctly

When working with Pandas, it's very useful to have columns which contains dates in a specific format called *datetime*. This allows for efficient manipulation and analysis of time-series data, such as sorting, filtering by date or time, and resampling for different time periods. Figure out which column(s) would be appropriate to convert to datetime, if any, and if so, perform the conversion to the correct format.

In [142...

```
df_spotify_0["endTime"] = pd.to_datetime(df_spotify_0["endTime"])
```

1.5 Unique artists

Find how many unique artists are in the dataset.

In [143...

```
len(list(set(df_spotify_0["artistName"])))
```

Out[143...

495

1.6 Unique songs

Find how many unique songs are in the dataset.

In [144...

```
len(list(set(df_spotify_0["trackName"])))
```

Out[144...

1308

Part 1: Questions

Q1: Which columns are in the dataset?

Q2: What timeframe does the dataset span?

Q3: How many unique artists are in the dataset?

Q4: How many unique songs are in the dataset?

Q1

The columns of the dataset are: endTime, artistName, trackName and msPlayed

Q2

the dataset spans the timeframe from dec 3rd 2022 to jan 2nd. 2023

Q3

There are 495 unique artists in the dataset

Q4

There are 1308 unique songs in the dataset

Part 2: Working with all the data

2.0 Importing all the dataframes

In Task 1, you only worked with about a month worth of data. Now, you will work with over a year worth.

In the *spotify_data* folder, there is more than just one listening record. Load each of the 14 listening records into a dataframe (1 dataframe per listening record), and concatenate them together into one large dataframe named `df`.

In [145...

```
import os

streaminghist = os.listdir("spotify_data")
hists = []
streaminghist

for hist in streaminghist:
    hists.append(pd.read_csv(f"spotify_data/{hist}"))
```

```
df = pd.concat(hists)
```

2.1 Sorting by time

Datasets often aren't perfect. One example of an issue that could occur is that the time-based data might not be in chronological order. If this were to happen, the rows in your dataframe could be in the wrong order. To ensure this isn't an issue in your dataframe, you should sort the dataframe in chronological order, from oldest to newest.

```
In [146... df = df.sort_values("endTime")
df
```

```
Out[146...      endTime  artistName  trackName  msPlayed
0  2022-12-03 02:02  Cigarettes After Sex      Truly      30000.0
1  2022-12-03 02:02    Leonard Cohen  Take This Waltz - Paris Version      8210.0
2  2022-12-06 21:05    Vlad Holiday  So Damn Into You      37895.0
3  2022-12-06 21:05        Lorde      Team      8984.0
4  2022-12-06 21:05    Ariana Grande  Into You      1221.0
...      ...      ...      ...      ...
11964  2023-12-07 21:13    Lana Del Rey      Ride      1126.0
11969  2023-12-07 21:14    Ariana Grande    my hair      23757.0
11970  2023-12-07 21:14    Leonard Cohen  Thanks for the Dance      9317.0
11971  2023-12-07 21:17    The Vaccines  Your Love Is My Favourite Band      14661.0
9062      NaN    The Lumineers      Ophelia      371.0
```

167439 rows × 4 columns

2.2 Setting a timeframe

For this investigation, we are only interested in investigating listening patterns from **2023**. Remove any data not from **2023** from the DataFrame.

```
In [147... df["endTime"] = pd.to_datetime(df["endTime"])
df.sort_values("endTime")
df = df[df["endTime"].dt.year == 2023]
df
```

Out[147...

	endTime	artistName	trackName	msPlayed
10881	2023-01-01 01:17:00	Ariana Grande	7 rings	139.0
10882	2023-01-01 01:17:00	Ariana Grande	7 rings	487.0
10883	2023-01-01 01:17:00	Ariana Grande	positions	417.0
10884	2023-01-01 01:17:00	Peach Pit	Being so Normal	2205.0
10885	2023-01-01 01:17:00	Kelly Clarkson	Santa, Can't You Hear Me	278.0
...	...	...	...	...
11959	2023-12-07 21:13:00	Lana Del Rey	Young And Beautiful	3146.0
11964	2023-12-07 21:13:00	Lana Del Rey	Ride	1126.0
11969	2023-12-07 21:14:00	Ariana Grande	my hair	23757.0
11970	2023-12-07 21:14:00	Leonard Cohen	Thanks for the Dance	9317.0
11971	2023-12-07 21:17:00	The Vaccines	Your Love Is My Favourite Band	14661.0

156557 rows × 4 columns

2.3 Deleting rows

Often in Data Science, you will encounter when a row entry has the value *NaN*, indicating missing data. These entries can skew your analysis, leading to inaccurate conclusions. For this task, identify and remove any rows in your DataFrame that contain NaN values.

Later in the course, you might encounter other techniques of dealing with missing data, typically referred to as *data imputation*. Here, though, you are just supposed to delete the entire rows with missing data.

In [148...

```
df = df.dropna()
df
```

Out[148...

	endTime	artistName	trackName	msPlayed
10881	2023-01-01 01:17:00	Ariana Grande	7 rings	139.0
10882	2023-01-01 01:17:00	Ariana Grande	7 rings	487.0
10883	2023-01-01 01:17:00	Ariana Grande	positions	417.0
10884	2023-01-01 01:17:00	Peach Pit	Being so Normal	2205.0
10885	2023-01-01 01:17:00	Kelly Clarkson	Santa, Can't You Hear Me	278.0
...	...	...	...	...
11959	2023-12-07 21:13:00	Lana Del Rey	Young And Beautiful	3146.0
11964	2023-12-07 21:13:00	Lana Del Rey	Ride	1126.0
11969	2023-12-07 21:14:00	Ariana Grande	my hair	23757.0
11970	2023-12-07 21:14:00	Leonard Cohen	Thanks for the Dance	9317.0
11971	2023-12-07 21:17:00	The Vaccines	Your Love Is My Favourite Band	14661.0

156539 rows × 4 columns

2.4 Convert from milliseconds to seconds

From `msPlayed`, create a new column `secPlayed` with the data converted from milliseconds to seconds. Then delete the column `msPlayed`.

In [149...

```
df["secPlayed"] = df["msPlayed"] / 1000
df = df.drop(["msPlayed"], axis=1)
df
```

C:\Users\lasse\AppData\Local\Temp\ipykernel_17132\4276919557.py:1: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df["secPlayed"] = df["msPlayed"] / 1000
```

Out[149...

	endTime	artistName	trackName	secPlayed
10881	2023-01-01 01:17:00	Ariana Grande	7 rings	0.139
10882	2023-01-01 01:17:00	Ariana Grande	7 rings	0.487
10883	2023-01-01 01:17:00	Ariana Grande	positions	0.417
10884	2023-01-01 01:17:00	Peach Pit	Being so Normal	2.205
10885	2023-01-01 01:17:00	Kelly Clarkson	Santa, Can't You Hear Me	0.278
...	...	...	...	...
11959	2023-12-07 21:13:00	Lana Del Rey	Young And Beautiful	3.146
11964	2023-12-07 21:13:00	Lana Del Rey	Ride	1.126
11969	2023-12-07 21:14:00	Ariana Grande	my hair	23.757
11970	2023-12-07 21:14:00	Leonard Cohen	Thanks for the Dance	9.317
11971	2023-12-07 21:17:00	The Vaccines	Your Love Is My Favourite Band	14.661

156539 rows × 4 columns

2.5 Finding top 10 favorite artists

Find the top `ten` artists with the highest total play time (in seconds). Plot your findings in a bar graph.

(hint: start by creating a new DataFrame with only `artistName` and your time column. To proceed, you will also likely need the `groupby` command from Pandas.)

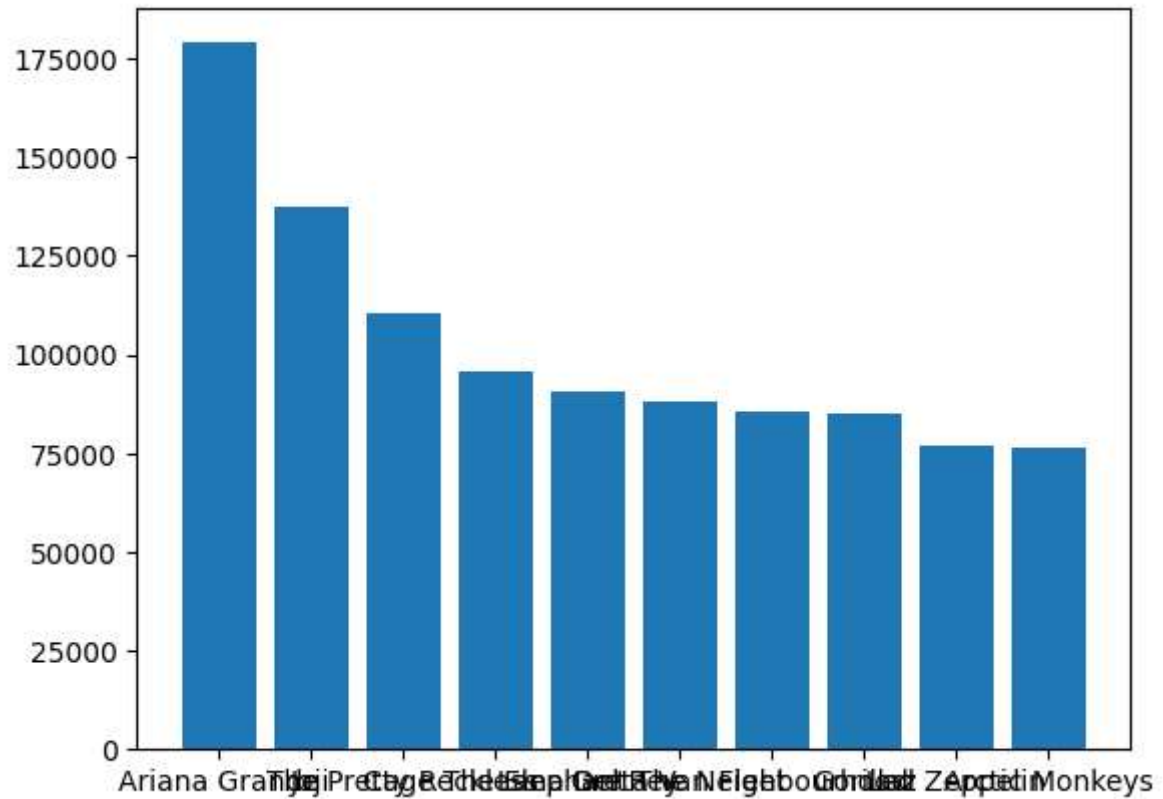
In [150...

```

dftopartists = df[["artistName", "secPlayed"]]
dftopartists = dftopartists.groupby("artistName", as_index=False).sum()
dftopartists = dftopartists.sort_values("secPlayed", ascending=False)
dftopartists = dftopartists.head(10)

plt.bar(x=dftopartists["artistName"], height=dftopartists["secPlayed"])
plt.show()

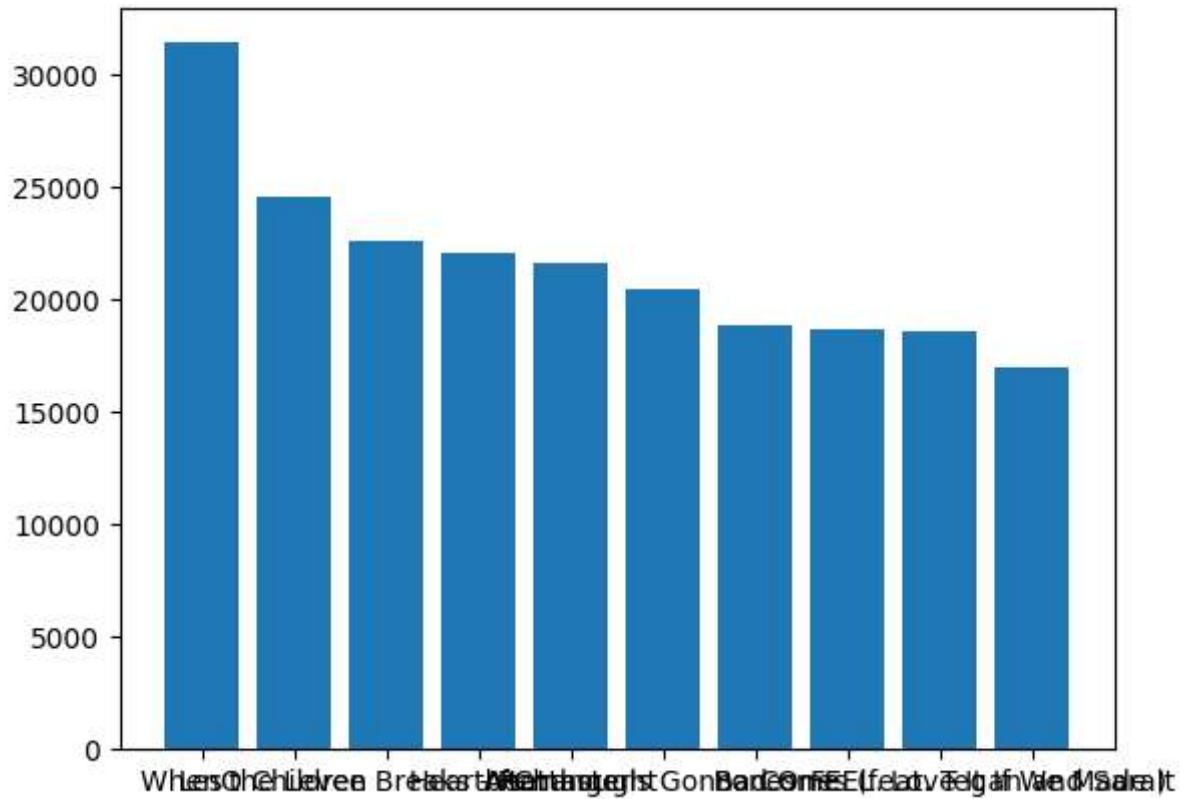
```



2.6 Finding top 10 favorite songs

Find the top `ten` songs with the highest play time. Create a graph visualizing the results.

```
In [151... dftopsongs = df[["trackName", "secPlayed"]]
dftopsongs = dftopsongs.groupby("trackName", as_index=False).sum()
dftopsongs = dftopsongs.sort_values("secPlayed", ascending=False)
dftopsongs = dftopsongs.head(10)
dftopsongs
plt.bar(x=dftopsongs["trackName"], height=dftopsongs["secPlayed"])
plt.show()
```



Part 3: Further analysis

3.0 Average listening time by hour

Generate a plot that displays the average amount of time that music is played for each hour of the day.

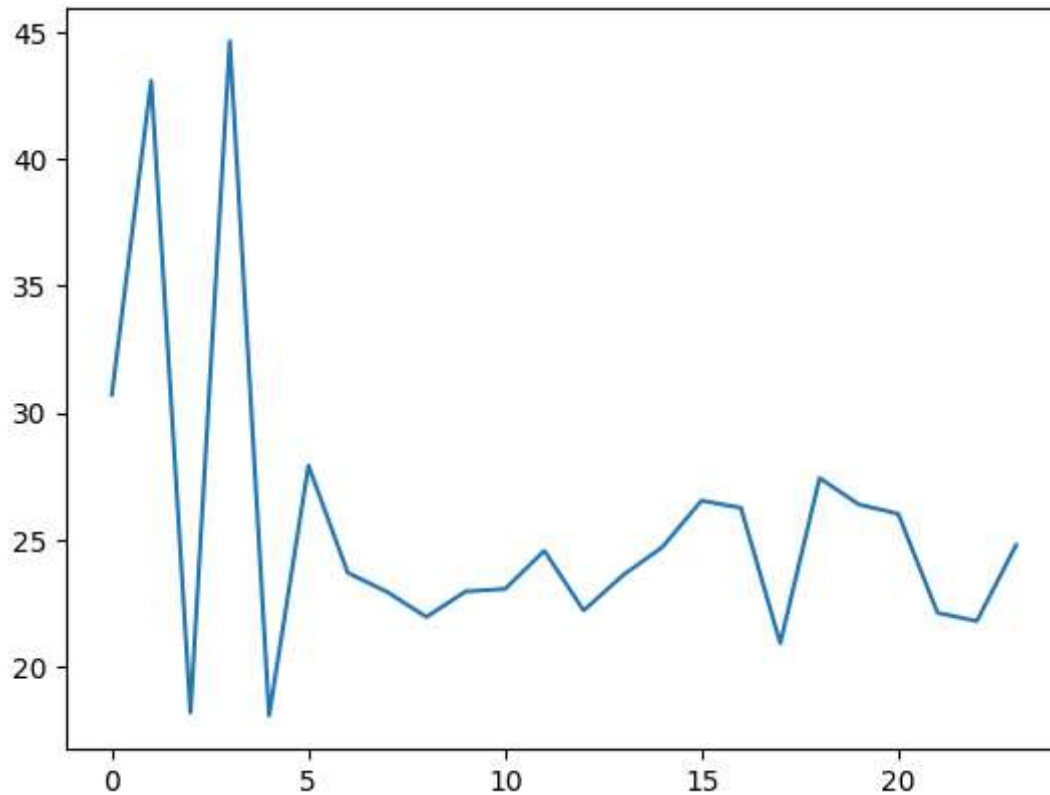
```
In [152... df["endTime"] = pd.to_datetime(df["endTime"])

h_avg = df.groupby(df["endTime"].dt.hour)["secPlayed"].mean()

fig1, ax1 = plt.subplots()

ax1.plot(h_avg)
```

```
Out[152... [<matplotlib.lines.Line2D at 0x17ea598c550>]
```



3.1 Morning music and evening music

I think many people find that some types of music are more suitable for morning listening and some music is more suitable for evening listening. Create a plot that compares the play time of the artists *Leonard Cohen* and *Rage Against the Machine* on an hour-by-hour basis. See if there are any differences.

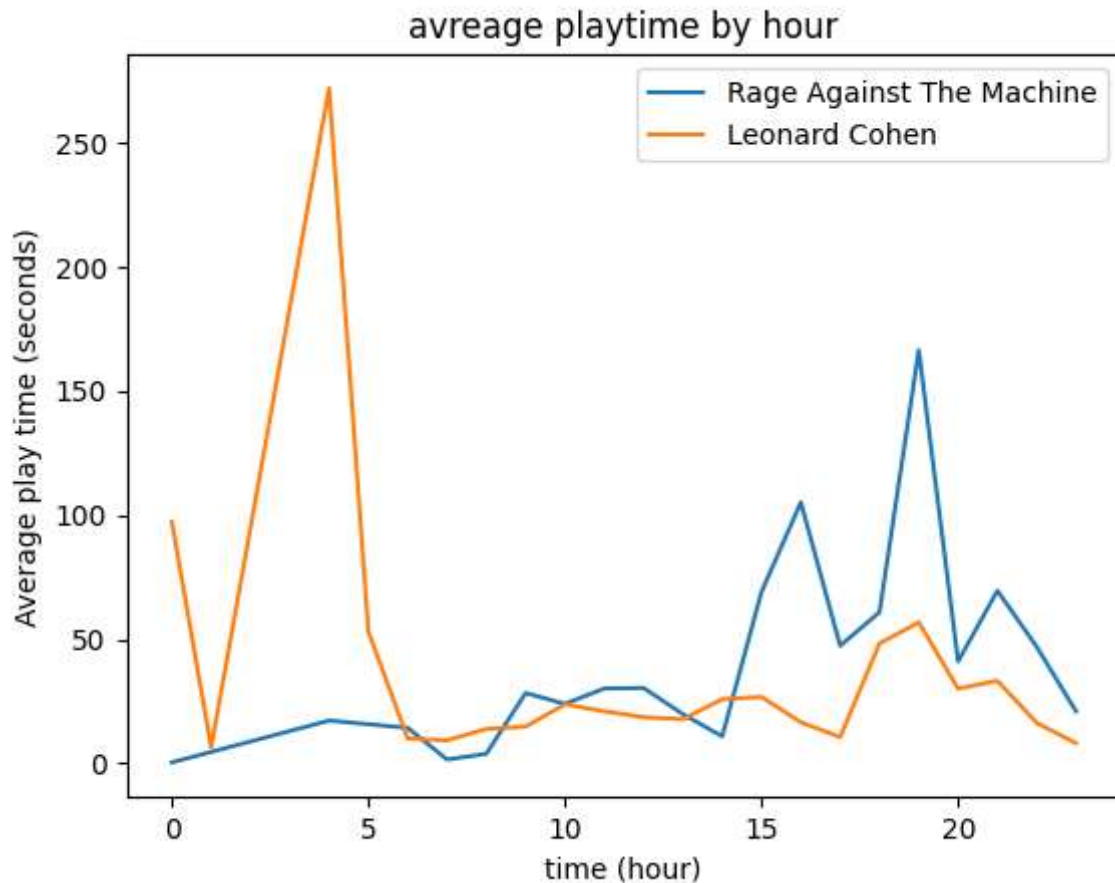
```
In [153... artists = ["Rage Against The Machine", "Leonard Cohen"]

fig2, ax2 = plt.subplots()

for artist in artists:
    df_artist = df[df["artistName"] == artist]
    h_avg = df_artist.groupby(df_artist["endTime"].dt.hour)["secPlayed"].mean()
    ax2.plot(h_avg, label=artist)

ax2.set_xlabel("time (hour)")
ax2.set_ylabel("Average play time (seconds)")
ax2.set_title("average playtime by hour")
ax2.legend()

plt.show()
```



3.2 Analysing skipped songs

Determining whether a song was skipped or listened to can be challenging. For this analysis, we'll simplify by defining a skipped song as any track played for less than 30 seconds.

Conversely, a song played for 30 seconds or more is considered listened to.

Add a column to your DataFrame to reflect this criteria: set the value to 1 if the song was played for less than 30 seconds (indicating a skipped song), and 0 if it was played for 30 seconds or longer.

```
In [154... df["skipped"] = (df["secPlayed"]/30 < 1).astype(int)
df.head()
```

```
Out[154... 
```

	endTime	artistName	trackName	secPlayed	skipped
10881	2023-01-01 01:17:00	Ariana Grande	7 rings	0.139	1
10882	2023-01-01 01:17:00	Ariana Grande	7 rings	0.487	1
10883	2023-01-01 01:17:00	Ariana Grande	positions	0.417	1
10884	2023-01-01 01:17:00	Peach Pit	Being so Normal	2.205	1
10885	2023-01-01 01:17:00	Kelly Clarkson	Santa, Can't You Hear Me	0.278	1

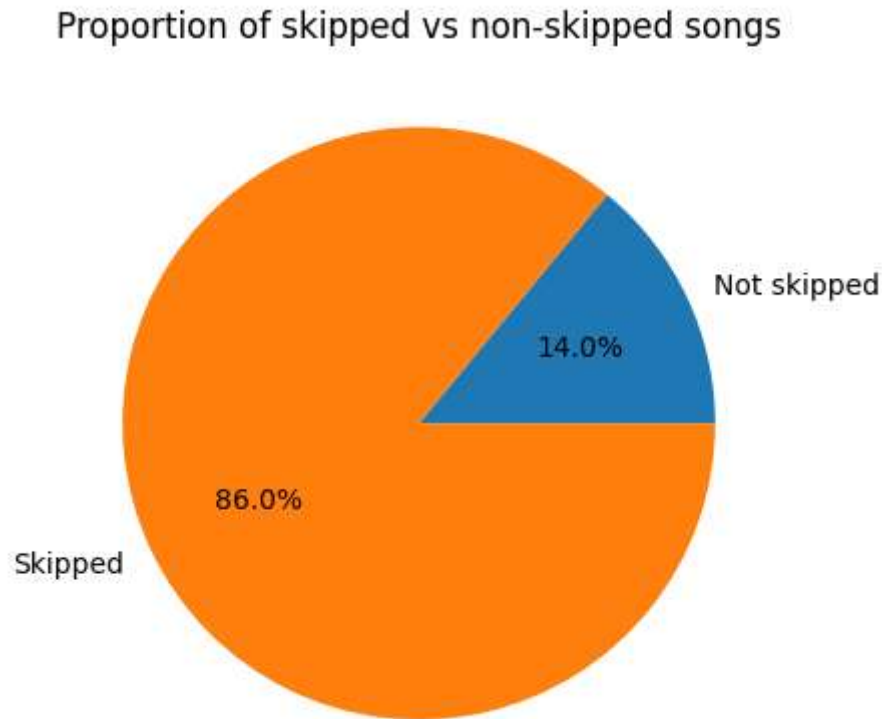
3.3 Plotting skipped songs

Create a pie-chart that compares amount of skipped songs to amount of non-skipped songs.

```
In [155... skip_counts = df["skipped"].value_counts().sort_index()

fig3, ax3 = plt.subplots()
ax3.pie(skip_counts, labels=["Not skipped", "Skipped"], autopct="%1.1f%%")
ax3.set_title("Proportion of skipped vs non-skipped songs")

plt.show()
```



3.4 Artists by percentage of songs skipped

For each artist in the dataset, calculate which percentage of their songs was skipped. Store this information in a new DataFrame called `df_skipped`. Store the percentage of skipped songs in a new column named `SkipRate`.

Example: If an artist has **100** songs in your dataset and **25** of these were skipped, the percentage of skipped songs for this artist would be $\frac{25}{100} = 25\%$

```
In [156... df_skipped = df.groupby("artistName")["skipped"].mean().reset_index()
df_skipped = df_skipped.rename(columns={"skipped": "SkipRate"})
df_skipped["SkipRate"] = df_skipped["SkipRate"] * 100

df_skipped.head()
```

Out[156...

	artistName	SkipRate
0	10cc	67.857143
1	2Pac	86.159844
2	3 Doors Down	50.000000
3	4 Non Blondes	72.131148
4	50 Cent	67.857143

3.5 Comparing artists by skip-rate

Find the **three** top artists with the lowest skip-rate and the **three** with the highest. Print their names, along with their skip-rate.

In [157...

```
lowest_skiprate = df_skipped.nsmallest(3, "SkipRate")
highest_skiprate = df_skipped.nlargest(3, "SkipRate")

print("Top 3 artists with the lowest skip-rate:")
print(lowest_skiprate[["artistName", "SkipRate"]])

print()
print("Top 3 artists with the highest skip-rate:")
print(highest_skiprate[["artistName", "SkipRate"]])
```

Top 3 artists with the lowest skip-rate:

	artistName	SkipRate
305	Gloria Gaynor	0.000000
645	Roc Boyz	11.111111
437	LACES	14.285714

Top 3 artists with the highest skip-rate:

	artistName	SkipRate
7	A Problem Squared	100.0
16	Acid Ghost	100.0
25	Albert Hammond Jr	100.0

Part 4: God Is a Data Scientist - The Ariana Deep-Dive

4.0 Ariana-DataFrame:

Create a new DataFrame called *df_ariana*, containing only rows with music by Ariana Grande.

In [158...

```
df_ariana = df[df["artistName"] == "Ariana Grande"]
df_ariana
```

Out[158...

	endTime	artistName	trackName	secPlayed	skipped
10881	2023-01-01 01:17:00	Ariana Grande	7 rings	0.139	1
10882	2023-01-01 01:17:00	Ariana Grande	7 rings	0.487	1
10883	2023-01-01 01:17:00	Ariana Grande	positions	0.417	1
10887	2023-01-01 01:17:00	Ariana Grande	Santa Baby	12.293	1
10888	2023-01-01 01:17:00	Ariana Grande	Right There (feat. Big Sean)	22.929	1
...	...	...	...	...	...
11948	2023-12-07 17:46:00	Ariana Grande	Almost Is Never Enough	28.483	1
11955	2023-12-07 20:51:00	Ariana Grande	needy	26.220	1
11961	2023-12-07 21:13:00	Ariana Grande	pete davidson	0.603	1
11968	2023-12-07 21:13:00	Ariana Grande	off the table (with The Weeknd)	13.448	1
11969	2023-12-07 21:14:00	Ariana Grande	my hair	23.757	1

19337 rows × 5 columns

4.1 Average skip rate

Create a histogram of the distribution of the skip-rate values of the different artists in your DataFrame `df_skipped`, with skip rates on one axis and number of artists on the other.

Then, retrieve the skip rate for Ariana Grande from your DataFrame `df_skipped`. Run the code in the cell below. Where on this distribution does Ariana Grande fall? Do I skip her songs more than average, or less?

In [159...

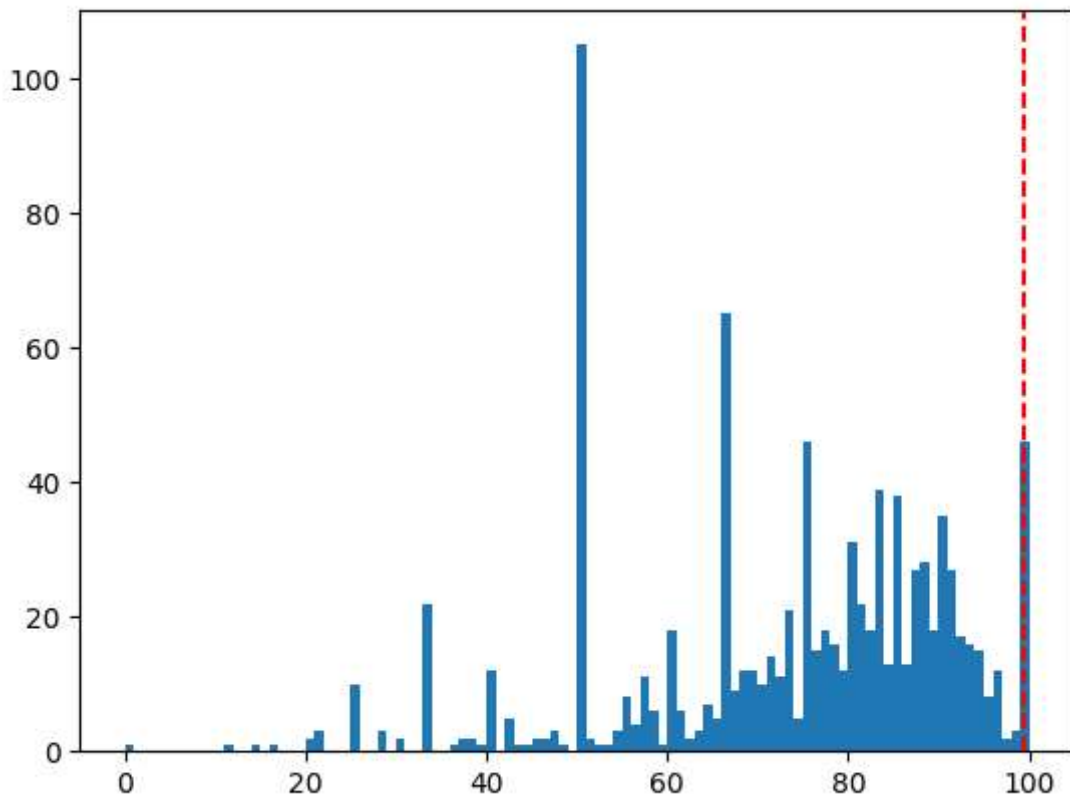
```

ariSkipdf = df_skipped[df_skipped["artistName"]=="Ariana Grande"]
ariSkip = ariSkipdf["SkipRate"].values[0]
print(ariSkip)

plt.hist(df_skipped["SkipRate"], bins=100)
plt.axvline(ariSkip, color="red", linestyle="--")
plt.show()

```

99.52939959662822



Part 4: Questions

Q1: Did I skip a lot of Ariana Grande's songs, or did I not, compared to the rest of the dataset?

Q2: What might be some possible reasons for Ariana Grande to be my nr.1 artist?

Q1

Yes you skipped over 99% of Ariana grande songs, which compared to the rest of the dataset is alot

Q2

A possible reason for Ariana grande being your top artist is that you have played a lot of ariana grande songs, so even though you skipped almost everytime, the small amounts compounded making ariana grande your top artist.